

Лабораторная работа №2

Тема: Работа с Docker.

Цель: Познакомиться с возможностями и получить практические навыки работы с Docker.

Ссылки:

1. <https://docs.docker.com>
2. <https://hub.docker.com/>
3. <https://doka.guide/tools/dockerfile/>
4. <https://k21academy.com/docker-kubernetes/docker-storage/>
1. <https://k21academy.com/docker-kubernetes/docker-tutorial>
2. <https://k21academy.com/docker-kubernetes/docker-networking-different-types-of-networking-overview-for-beginners/>
3. <https://ru.bitdegree.org/rukovodstvo/docker-dlja-novichkov/>
4. <https://habr.com/ru/companies/slurm/articles/528206/>
5. <https://itisgood.ru/2019/09/10/docker-setevoe-vzaimodejstvie-101/>
6. <https://itisgood.ru/2019/10/29/objasnenie-koncepcii-setej-v-docker/>
7. <https://www.tune-it.ru/web/adpashnin/blog/-/blogs/docker-network>

Для защиты ЛР необходимо

- Оформить Отчет со скринами кода в командной строке и результатами его выполнения
- Продемонстрировать работу с командами docker: build, tag, run, start, stop, pause, unpause, restart, ps, logs, ps, images, network, volumes, inspect, а также знать назначение инструкций в Dockerfile и Docker Compose

Задание:

1. Подготовьте рабочее окружение в соответствии с типом вашей операционной системы
 - Установите Docker
 - Выполните базовую настройку
2. Изучите простейшие консольные команды и возможности Docker Desktop (см. лекцию), создать собственный контейнер docker/getting-started, открыть в браузере и изучить tutorial

```
[sunieq@MacBook-Air-Vladislav LR2 % docker run -d -p 80:80 docker/getting-started
Unable to find image 'docker/getting-started:latest' locally
latest: Pulling from docker/getting-started
14f901bbf056: Pull complete
fa3f58a317be: Pull complete
476bb2a1cc22: Pull complete
a60aada4c44a: Pull complete
a879581b8e12: Pull complete
261da4162673: Pull complete
2f61404bb4b8: Pull complete
d0193f05f10f: Pull complete
33a28b928e89: Pull complete
Digest: sha256:d79336f4812b6547a53e735480dde67f8f8f7071b414fbd9297609ffb989abc1
Status: Downloaded newer image for docker/getting-started:latest
59ec584680580d12a429f6beefbc29daabe1d8f4d830e9c00bf80a37e0216b6f
```

3. Создайте docker image, который запускает скрипт с использованием функций из https://github.com/smartigaorg/geometric_lib.

- a. Данные необходимые для работы скрипта передайте любым удобным способом (например: конфиг файл через docker volume, переменные окружения, перенаправление ввода). Изучите простейшие консольные команды для работы с docker(см. лекцию). Зарегистрируйтесь на DockerHub и выберите необходимые для проекта образы
- b. Создать Dockerfile для реализации сборки собственных Docker образов

```
Dockerfile
1 FROM python:3.11-slim
2
3 RUN apt-get update && apt-get install -y git
4
5 WORKDIR /app
6
7 RUN git clone https://github.com/smartigaorg/geometric_lib.git
8
9 COPY app.py .
10
11 ENV PYTHONPATH=/app
12
13 CMD ["python", "app.py"]
```

```
[sunieq@MacBook-Air-Vladislav LR2 % docker build -t geometric-app .
```

- c. Использовать его для создания контейнера. Протестировать использование контейнера

```
[sunieq@MacBook-Air-Vladislav LR2 % docker run --rm -e SHAPE=square -e VALUE=5 geometric-app
Square:
Area: 25.0
Perimeter: 20.0
```

4. Скачать любой доступный проект с GitHub с произвольным стеком технологий (пример – см. индивидуальное задание) или использовать свой, ранее разработанный. Создать для него необходимый контейнер, используя Docker Compose для управления многоконтейнерными приложениями. Запустить проект в контейнере.

```
[sunieq@MacBook-Air-Vladislav React-Redux-Node-Mongodb % docker compose up --build -d
```

(Примеры Images: https://hub.docker.com/_/phpmyadmin,
https://hub.docker.com/_/mysql, https://hub.docker.com/_/postgres)

5. Настроить сети и тома для обеспечения связи между контейнерами и сохранения данных (исходные данные, логин, пароль и т.д.)

```

React-Redux-Node-Mongodb > compose.yaml
1  services:
2    mongo:
3      image: mongo:6
4      container_name: rrnm_mongo
5      networks:
6        - appnet
7      volumes:
8        - mongo_data:/data/db
9
10   app:
11     build: .
12     container_name: rrnm_app
13     ports:
14       - "3000:3000"
15     environment:
16       - MONGO_URI=mongodb://mongo:27017/mydb
17     depends_on:
18       - mongo
19     networks:
20       - appnet
21
22   networks:
23     appnet:
24       driver: bridge
25
26   volumes:
27     mongo_data:

```

6. Разместите результат в созданный репозиторий в DockerHub

```
docker push sunieqqq/react-redux-node-mongodb-app:1.0
```

7. Выполните следующие действия с целью изучить особенности сетевого взаимодействия:

- Получить информацию о всех сетях, работающих на текущем хосте и подробности о каждом типе сети

```
[sunieq@MacBook-Air-Vladislav React-Redux-Node-Mongodb % docker network ls]
NETWORK ID          NAME                DRIVER              SCOPE
ad218f4d2f7f        bridge              bridge              local
761d0de3cb4b        host                host                local
89685ed4fc6b        none                null                local
6968c1b0de7b        react-redux-node-mongodb_appnet  bridge              local
1068513392d7        react-redux-node-mongodb_default  bridge              local
```

```
docker network inspect react-redux-node-mongodb_appnet
```

```
docker network inspect bridge
```

```
docker network inspect none
```

- Создать свою собственную сеть bridge, проверить, создана ли она, запустить Docker-контейнер в созданной сети, вывести о ней всю информацию(включая IP-адрес контейнера), отключить сеть от контейнера

```
docker network create --driver bridge my_bridge_network
```

```
sunieq@MacBook-Air-Vladislav React-Redux-Node-Mongodb % docker network ls
NETWORK ID          NAME                DRIVER              SCOPE
ad218f4d2f7f        bridge              bridge               local
761d0de3cb4b        host                host                 local
29dbb1f8afa9        my_bridge_network   bridge               local
89685ed4fc6b        none                null                 local
6968c1b0de7b        react-redux-node-mongodb_appnet bridge               local
1068513392d7        react-redux-node-mongodb_default bridge               local

sunieq@MacBook-Air-Vladislav React-Redux-Node-Mongodb % docker run -d --name test_container --network my_bridge_network nginx
Unable to find image 'nginx:latest' locally
latest: Pulling from library/nginx
f2c05cdfb149: Pull complete
2668e3434976: Pull complete
e88d7844c33d: Pull complete
901e94f777d1: Pull complete
4a89256e588a: Pull complete
c813174c999b: Pull complete
3c9c97ab7d80: Download complete
35dee7ece046: Download complete
Digest: sha256:0236ee02dcbce00b9bd83e0f5fbc51069e7e1161bd59d99885b3ae1734f3392e
Status: Downloaded newer image for nginx:latest
75948f0816d0861d4dc7671920faa4b22037b10d7c9b5e6f44ac0b5d1cae0161
```

```
docker network inspect my_bridge_network
```

```
docker network disconnect my_bridge_network test_container
```

- Создать еще одну сеть bridge, вывести о ней всю информацию, запустить в ней три контейнера, подключиться к любому из контейнеров и пропинговать два других из оболочки контейнера, убедиться, что между контейнерами происходит общение по IP-адресу

```
docker network create --driver bridge my_bridge2
```

```
docker run -d --name c1 --network my_bridge2 alpine sleep 1000000
```

```
docker run -d --name c2 --network my_bridge2 alpine sleep 1000000
```

```
docker run -d --name c3 --network my_bridge2 alpine sleep 1000000
```

```
docker exec -it c1 sh
```

```
[/ # ping -c 3 c2
```

```
PING c2 (172.21.0.3) 56(84) bytes of data.
64 bytes from c2.my_bridge2 (172.21.0.3): icmp_seq=1 ttl=64 time=2.13 ms
64 bytes from c2.my_bridge2 (172.21.0.3): icmp_seq=2 ttl=64 time=0.148 ms
64 bytes from c2.my_bridge2 (172.21.0.3): icmp_seq=3 ttl=64 time=0.148 ms
```

```
[/ # ping -c 3 c3
```

```
PING c3 (172.21.0.4) 56(84) bytes of data.
64 bytes from c3.my_bridge2 (172.21.0.4): icmp_seq=1 ttl=64 time=1.67 ms
64 bytes from c3.my_bridge2 (172.21.0.4): icmp_seq=2 ttl=64 time=0.209 ms
64 bytes from c3.my_bridge2 (172.21.0.4): icmp_seq=3 ttl=64 time=0.247 ms
```

- Создать свою собственную сеть overlay, проверить, создана ли она, вывести о ней всю информацию

```
docker swarm init
```

```
docker network create -d overlay my_overlay_network
```

```
docker network inspect my_overlay_network
```

- Создать еще одну сеть overlay, проверить, создана ли она, вывести о ней всю информацию, удалить сеть

```
docker network create -d overlay my_overlay2
```

```
sunieq@MacBook-Air-Vladislav React-Redux-Node-Mongodb % docker network ls
NETWORK ID          NAME                DRIVER              SCOPE
ad218f4d2f7f        bridge              bridge              local
3e35d3f7790f        docker_gwbridge     bridge              local
761d0de3cb4b        host                host                local
rx019obboqpo        ingress             overlay             swarm
3ad9a4bd0162        my_bridge2          bridge              local
29dbb1f8afa9        my_bridge_network   bridge              local
k06i89tm4hwm        my_overlay2         overlay             swarm
vm33779m5t4n        my_overlay_network  overlay             swarm
89685ed4fc6b        none                null                local
6968c1b0de7b        react-redux-node-mongodb_appnet  bridge              local
1068513392d7        react-redux-node-mongodb_default  bridge              local
```

```
docker network inspect my_overlay2
```

```
docker network rm my_overlay2
```

```
sunieq@MacBook-Air-Vladislav React-Redux-Node-Mongodb % docker network ls
NETWORK ID          NAME                DRIVER              SCOPE
ad218f4d2f7f        bridge              bridge              local
3e35d3f7790f        docker_gwbridge     bridge              local
761d0de3cb4b        host                host                local
rx019obboqpo        ingress             overlay             swarm
3ad9a4bd0162        my_bridge2          bridge              local
29dbb1f8afa9        my_bridge_network   bridge              local
vm33779m5t4n        my_overlay_network  overlay             swarm
89685ed4fc6b        none                null                local
6968c1b0de7b        react-redux-node-mongodb_appnet  bridge              local
1068513392d7        react-redux-node-mongodb_default  bridge              local
```

- Попробовать создать сеть host, сохранить результат в отчет.

```
docker network create -d host my_host_network
```

```
Error response from daemon: only one instance of "host" network is allowed
```

Индивидуальное задание

Вар-т	Условие – стек технологий
1.	ReactJs-Django-SQLitea
2.	ReactJS-Django-MySQL
3.	PHP, Django REST framework, MySQL and React
4.	Django 5, React, Bootstrap 5 with Python 3, PostgreSQL
5.	React, Node , PostgreSQL
6.	React.js - Redux - Node.js - Mongodb - Webpack
7.	Mongo DB, Express, Angular, Node.js
8.	Linux, Apache, MySQL, PHP

9.	MongoDB, Express.js, AngularJS, Node.js
10.	Ruby, Rails, Rack, Passenger
11.	.NET, MS SQL Server
12.	Java, MySQL
13.	ASP.NET MVC, IIS, Microsoft Azure, SQL Server
14.	React.js - Redux - Node.js - MongoDB
15.	ReactJS-Django-SQLite
16.	ReactJS-Django-MySQL
17.	PHP, Django REST framework, MySQL and React
18.	Django 5, React, Bootstrap 5 with Python 3, PostgreSQL
19.	React, Node , PostgreSQL
20.	React.js - Redux - Node.js - MongoDB - Webpack
21.	Mongo DB, Express, Angular, Node.js
22.	Linux, Apache, MySQL, PHP
23.	MongoDB, Express.js, AngularJS, Node.js
24.	Ruby, Rails, Rack, Passenger
25.	.NET, MS SQL Server
26.	Java, MySQL
27.	ASP.NET MVC, IIS, Microsoft Azure, SQL Server
28.	React.js - Redux - Node.js - MongoDB
29.	Linux, Apache, MySQL, PHP
30.	MongoDB, Express.js, AngularJS, Node.js

Контрольные вопросы

1. Что представляет собой контейнеризация?
2. Что такое Docker?
3. Что является реестром Docker?
4. Что такое Docker Engine?
5. Как происходит связь между клиентом Docker и демоном Docker?
6. Для чего необходим образ контейнера?
7. Что представляет собой контейнер Docker?
8. Каков метод создания Docker-контейнера?
9. Каковы этапы жизненного цикла контейнера Docker?
10. В каких средах можно использовать контейнерные приложения?
Что такое виртуальная машина?
11. Является ли Docker VM-технологией?
12. Какие различия между виртуальными машинами и контейнерами Docker?
13. Зачем использовать контейнеры?
14. Что выдаст команда \$ docker ps ?
15. Сколько идентификаторов по умолчанию имеет каждый контейнер?
16. Как задается имя контейнера?
17. Для чего нужна команда container start?
18. Как создать контейнер без запуска?
19. Что такое запуск контейнеров в интерактивном режиме?
20. Что представляет собой образ?
21. Что такое Dockerfile?
22. Для чего необходим Docker Compose?
23. Какая структура файла Compose?

24. Как запустить несколько контейнеров, используя один сервис?
25. Что такое предоставление порта связанным службам?
26. Для чего используются тома?
27. Для чего применяют переменные среды?
28. Как определяют правила связи между контейнерами?
Написать файл Docker для создания и копирования каталога и построить его с использованием модулей Python?
29. Какие сети доступны по умолчанию в Docker?
30. Приведите необходимые шаги для развертывания докеризированного приложения, сохраненного в репозитории Git